COMP1531
Software Engineering Fundamentals

Week 2

Development - Multi-File & Importing

In this Lecture

Why?

Most software projects involve working across multiple files that need to interact

Big projects ≠ one file

What?

- Import Libraries
- Importing our own files

Importing Libraries

Similar to C, NodeJS (Javascript) has a number of . These libraries come with the interpret and don't need to be installed. These libraries were written by other programmers.

Examples of importing are below.

C

```c
#include <stdio.h>
```

Javascript

```javascript
import fs from 'fs';
```

Importing Libraries

How we use them is different too.

C

```c
#include <stdio.h>

int main() {
  printf("Hello\n");
}
```

Javascript

```javascript
import path from 'path';
console.log(path.resolve('./'));
```

Importing Our Own Files

We're also able to import our own files into another file, allowing us to separate out logic!

```
function manyString(repeat, str) {
  let outString = '';
  for (let i = 0; i < repeat; i++) {
    outString += str;
  }
  return outString;
}

console.log(manyString(5, 'hello '));
```

split_before.js

Importing Our Own Files

We've now split the code up into two files.

```js
function manyString(repeat, str) {
  let outString = '';
  for (let i = 0; i < repeat; i++) {
    outString += str;
  }
  return outString;
}

export default manyString;
```

split_after_lib.js

```js
import manyString_hello from './split_after_lib.js';

console.log(manyString_hello(5, 'hello '));
```

split_after_main.js

We use export default X when we want to export a single thing.

Importing Our Own Files

We've now split the code up into two files.

```
function manyString(repeat, str) {
  let outString = '';
  for (let i = 0; i < repeat; i++) {
    outString += str;
  }
  return outString;
}

export default manyString;
```

split_after_lib.js

```
import manyString_hello from './split_after_lib.js';

console.log(manyString_hello(5, 'hello '));
```

split_after_main.js

- We use export default X when we want to export a single thing.
- But now what if we want to export multiple things?

Importing Our Own Files

For example, how do we export both functions from this file?

```
function manyString(repeat, str) {
  let outString = '';
  for (let i = 0; i < repeat; i++) {
    outString += str;
  }
  return outString;
}

function addBrackets(string) {
  return `(${string})`;
}

export default manyString; // How do we add more??
```

multi_export_lib_p_1.js

Importing Our Own Files

Instead of exporting one thing, we can export many things! Please note, this is not exporting an object.

```
function manyString(repeat, str) {
  let outString = '';
  for (let i = 0; i < repeat; i++) {
    outString += str;
  }
  return outString;
}

function addBrackets(string) {
  return `(${string})`;
}

export {
  manyString,
  addBrackets,
};
```

multi_export_lib_p_2.js

Importing Our Own Files

We can use destructuring for importing as well.

```
function manyString(repeat, str) {
  let outString = '';
  for (let i = 0; i < repeat; i++) {
    outString += str;
  }
  return outString;
}

function addBrackets(string) {
  return `(${string})`;
}

export {
  manyString,
  addBrackets,
};
```

multi_export_lib_p_2.js

```
import { addBrackets, manyString as forum_hello} from './multi_export_lib_p

const b = addBrackets('hello ');
const many = forum_hello(5, b);
console.log(many);
```

multi_export_main_p_2.js

Importing Our Own Files

In summary:

Default Export

```
function a() {
  return 0;
}

export default a;
```

```
import a from './above.js'
```

Export one thing

Named Export

```
function a() {
  return 0;
}

function b() {
  return 0;
}

export {
  a,
  b,
};
```

```
import { a, b } from './above.js';
```

Export one or more things

Other Notes About Importing

In general we try and always use named exports. This:

1. Adds **flexibility to our code design** as it allows us to add more exports later.
2. Means that users of a library by default have to **reuse the same name** of your function.

```
function sum(a, b) {
  return a + b;
}

export default sum;
```

fns.js

```
import anyNameIWant from './fns.js';

console.log(anyNameIWant(1,2));
```

main.js

Other Notes About Importing

That being said, with named exports (i.e. an export within an object) we can still alias the name if we need to. This is especially useful if you already have functions in your file that use that name.

```
function sum(a, b) {
  return a + b;
}

export {
  sum,
};
```

fns.js

```
import { sum as simpleSum } from './fns.js';

function sum(a, b, c) {
  return a + b + c;
}

console.log(simpleSum(1,2));
```

main.js

Other Notes About Importing

And if we're importing too many things, we can just break it up over multiple lines.

```
import {
  function1,
  function2,
  function3,
  function4,
} from './bigfile.js';
```

Finally

In summary:

Feature	`export default`	`export { }`
How many allowed per file?	Only **one**	Unlimited
Import with braces?	❌ No	✅ Yes
Can rename on import?	✅ Yes	❌ Not unless using `as`
Best for	Main thing in file	Multiple utilities

Feedback

Or go to the [form here]